SBM CMIS RTOS Examples Documentation

Version

Table of Contents

Contents:

Ejemplos de uso de CMSIS RTOS V2 5

- Descarga del código 6
- Listado de ejemplos incluidos 7
- Configuración del Proyecto de Keil 8
  - Uso del simulador 8
  - Uso del hardware 9
  - Depuración de aplicaciones usando CMSIS-RTOS V2 9

ejemplothread : Uso básico de un thread en CMSIS RTOS v2 11

- Descripción General 12
- Estructura de Datos 13
- Inicialización de los Hilos 14
- Función del Hilo 15
- Uso de HAL y CMSIS RTOS 16
- Código Fuente 17
- Dependencias 19
- Preguntas y respuestas sobre **ejemplothread** 20
  - ¿Qué hace este código? 21
  - ¿Qué es la estructura *mygpio_pin* ? 21
  - ¿Cómo se inicializan los hilos? 21
  - ¿Qué hace la función *Thread()* ? 21
  - ¿Se ejecutan los hilos al mismo tiempo? 22
  - ¿Qué significa *osDelay()* ? 22
  - ¿Qué pasa si *osThreadNew()* devuelve NULL? 22
  - ¿Qué ficheros de cabecera se utilizan? 22

ejemplothreads : Uso básico de threads en CMSIS RTOS v2 23

- Descripción General 12
- Estructura de Datos 13
- Inicialización de los Hilos 14
- Función del Hilo 15
- Uso de HAL y CMSIS RTOS 16

- Código Fuente 17
- Dependencias 19
- Preguntas y respuestas sobre **ejemplothreads** 32
  - ¿Qué hace este código? 21
  - ¿Qué hace la función *Thread()* ? 21
  - ¿Se ejecutan los hilos al mismo tiempo? 22
  - ¿Qué pasa si *osThreadNew()* devuelve NULL? 22
  - ¿Qué ficheros de cabecera se utilizan? 22

ejemplothreads-queues : Uso básico de threads y colas en CMSIS RTOS v2 35

- Descripción General 12
- Estructura de Datos 13
- Inicialización de los Hilos 14
- Función del Hilo Producer 39
- Función del Hilo Consumer 40
- Uso de HAL y CMSIS RTOS 16
- Código Fuente 17
- Dependencias 19
- Preguntas y respuestas sobre **ejemplothreads-queues** 46
  - ¿Cuál es el propósito de la cola de mensajes *id_MsgQueue* en esta aplicación? 46
  - ¿Qué función cumple el bucle anidado en el hilo *Producer* ? 47
  - ¿Cuanto tiempo tarda en llenarse la cola de mensajes *id_MsgQueue* ? 47
  - ¿cuanto vale la variable errors_or_timeouts despues de 1 minuto de ejecución del código? 47

ejemplothreads-flags : Uso básico de threads y flags en CMSIS RTOS v2 48

- Descripción General 12
- Estructura de Datos 13
- Inicialización de los Hilos 14
- Función del Hilo Producer 39
- Función del Hilo Consumer 40
- Uso de HAL y CMSIS RTOS 16
- Código Fuente 17
- Dependencias 19
- Preguntas y respuestas sobre **ejemplothreads-flags** 59
  - Se modifica el código del Producer para que envíe ambas señales (0x0001 y 0x0002) de forma casi simultánea, seguido de un delay de 1 segundo: 59

CMSIS RTOS V2 Usage Examples 61
- Code download 62
- List of included examples 63
  - Keil Project Configuration 63
  - Using the simulator 63
  - Using the hardware 64
  - Debugging applications using CMSIS-RTOS V2 64

Ejemplos de uso de CMSIS RTOS V2

El repositorio contiene los ejemplos básicos para entender el funcionamiento de la API CMSIS-RTOS V2 utilizando el sistema operativo RTX version 5. Los ejemplos están implementados para utilizar mínimamente los periféricos del microcontrolador y hacer hincapié en los conceptos de manejo del Sistema Operativo.

Los ejemplos se han implementado para el STM32F429 utilizado en la asignatura **Sistemas Basados en Microprocesador** de la **(ETSI Sistemas de Telecomunicación) Universidad Politécnica de Madrid** y se pueden ejecutar utilizando el simulador del microprocesador incluido en el entorno de ARM keil Microvision o bien el hardware.

Descarga del código

Para descargar el código puede utilizar un cliente de git en su ordenador o bien descargar el repositorio completo (formato **zip**). Las instrucciones para clonar el repositorio son:

Note

Descarga del código

```
$ git clone https://github.com/mruizglz/SBM-rtos.git
```

Listado de ejemplos incluidos

Ejemplos incluidos

Carpeta	Objetivos
ejemplothreads	Aprender el manejo básico de creación de threads. Uso de la misma función con parámetros parea crear multiples threads
ejemplothreads-flags	Sincronización de threads usando flags
ejemplothreads-queues	Intercambio de datos entre threads usando colas
ejemplothreads-timers	Gestion de timers “software”

Configuración del Proyecto de Keil

Uso del simulador

ARM Keil Microvision dispone de opciones para configurar donde se ejecutará la aplicación (Icono *Options for Target*). Seleccione **Debug** y active el uso del simulador (**Use Simulator**). Es necesario que configure el fichero de inicialización (**Initialization File**) para que cargue un script de configuración del microcontrolador. En este caso, seleccione el fichero `simulator.ini` que se encuentra en cada una de las carpetas de ejmplo. Por ejemplo en la carpeta `.\ejemplothreads` del repositorio encontrará el fichero **simulador.ini** con este contenido:

```
MAP 0x40000000,0x400FFFFF read write
MAP 0xE0000000,0xE00FFFFF read write
```

El significado de estas instrucciones es habilitar para el simulador las operaciones de lectura/escritura en las zonas de memoria donde se encuentran los periféricos.

Tip

Cuando se usa el simulador de Keil las operaciones de escritura y lectura de los periféricos no tienen ningún efecto y por tanto no podrá simular el comportamiento hardware de los mismos Todas las operaciones de la capa HAL que actúan sobre periféricos no tendrán ningún efecto. Por ejemplo, si en el código se configura un pin como salida y luego se escribe un valor alto en el mismo, no podrá ver ningún cambio en el estado del pin.

Como podrá ver en el código del programa `main.c` existe compilación condicional para incluir o no el código de configuración del RCC para usar un reloj externo (HSE). Si utiliza el simulador debe desactivar esta opción y usar el reloj interno (HSI) que es el que utiliza el simulador. La pestaña **C/C++(AC6)** permite añadir en `define` etiquetas. Incluya `SIMULATOR` si quiere utilizar el simulator.

Uso del hardware

Si dispone de una placa con el microcontrolador STM32F429 puede ejecutar el código directamente en el hardware. En este caso debe configurar las opciones del proyecto para que utilice el ST-Link en lugar del simulador.

No defina la variable `SIMULATOR` en las opciones de compilación para que el circuito de RCC se configure adecuadamente.

Depuración de aplicaciones usando CMSIS-RTOS V2

La depuración de las aplicaciones se debe realizar combinando el uso de puntos de ruptura y de la aplicación RTX RTOS view disponible en el menu `View->Watch Windows->RTX RTOS` . Esta permite ver el estado en el que se encuentran los diferentes objetos del sistema operativo cuando el procesador pausa su ejecución. Herramientas complementarias para entender el funcionamiento de una aplicación son `Logyc Analyzer` , `Performance Analyzer` , `System Analyzer` , `Event Recorder` . `Event Statistics` y `Symbols Window`

Symbols Window

La opción **Symbols Window** permite visualizar y explorar todos los símbolos definidos en el proyecto, incluyendo variables globales, variables estáticas, funciones y direcciones de registros . Esta ventana es útil para depuración y análisis en tiempo real.

- Muestra una lista jerárquica de todos los símbolos disponibles en el programa cargado.
- Permite buscar y filtrar símbolos por nombre.
- Muestra la dirección y el valor actual de cada símbolo durante la sesión de depuración.
- Facilita el arrastre de variables a otras ventanas de análisis, como el Watch Window o el Logic Analyzer.
- Permite examinar variables optimizadas si están disponibles en la tabla de símbolos.
- Si un símbolo no aparece, verifique la configuración de optimización del compilador y el ámbito de la variable.

Para utilizarlo:

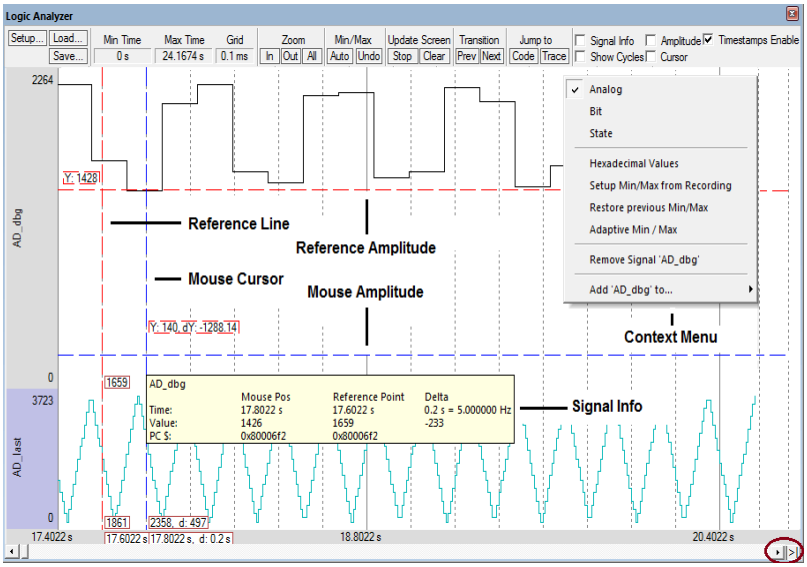
1. Iniciar una sesión de depuración.
2. Abrir la ventana desde el menú: View ▸ Symbol Window .

3. Buscar el símbolo deseado utilizando el campo de filtro.

4. Arrastrar el símbolo a la ventana de Watch o `Logic Analyzer` para su monitorización.

Logic Analyzer

Permite visualizar la evolución temporal de variables que sean globales a la aplicación, el contenido de posiciones de memoria, etc. Se puede configurar el rango de valores y es muy apropiado para comparar visualmente la evolución de la aplicacón software a través del seguimiento de variables. Para agregar señales al `Logic Analyzer` puede arrastrarlas de la ventana de símbolos o escribir el nombre de la misma.



Analizador lógico de ARM Keil Microvision.

ejemplothread : Uso básico de un thread en CMSIS RTOS v2

Este documento describe el funcionamiento de un programa en C que utiliza CMSIS RTOS v2 y la biblioteca HAL de STM32 para controlar dos LEDs mediante hilos concurrentes.

Descripción General

El programa crea un hilo LEDs conectado al pin PB0 del microcontrolador STM32F4.El hilo alterna el estado de su LED con una frecuencia distinta, utilizando funciones del sistema operativo en tiempo real (RTOS) y la biblioteca HAL para la configuración y manipulación de los pines GPIO.

Estructura de Datos

Se define una estructura llamada `mygpio_pin` que encapsula toda la información necesaria para controlar un LED:

- `GPIO_InitTypeDef pin` : configuración del pin (modo, velocidad, tipo de salida).
- `GPIO_TypeDef *port` : puerto GPIO al que pertenece el pin.
- `int delay` : retardo en milisegundos entre cada cambio de estado del LED.
- `uint8_t counter` : contador que se alterna en cada iteración del hilo.

Esta estructura permite pasar todos los parámetros necesarios a la función del hilo de forma organizada.

Inicialización de los Hilos

La función `Init_Thread` realiza las siguientes tareas:

1. Habilita el reloj del puerto GPIOB.
2. Configura `mygpio_pin` para el pin PB0.
3. Crea un hilo con `osThreadNew` , que ejecuta la función `Thread` pasandole una estructura `mygpio_pin` para el pin PB0.

Función del Hilo

La función `Thread` realiza lo siguiente:

1. Inicializa el pin GPIO usando `HAL_GPIO_Init` .
2. Entra en un bucle infinito donde: - Alterna el valor del contador con `~counter` . - Cambia el estado del pin con `HAL_GPIO_TogglePin` . - Espera el tiempo definido en `delay` usando `osDelay` .

Esto provoca que el LED conectado al pin correspondiente parpadee con una frecuencia determinada.

Uso de HAL y CMSIS RTOS

- **HAL (Hardware Abstraction Layer)** : se utiliza para configurar e inicializar los pines GPIO de forma sencilla y portable.
- **CMSIS RTOS v2** : proporciona las funciones para crear y gestionar hilos, como `osThreadNew` y `osDelay` .

Código Fuente

```
#include "cmsis_os2.h"
#include "stm32f4xx_hal.h"
#include <stdlib.h>

osThreadId_t tid_Thread;

GPIO_InitTypeDef led_ld1 = {
    .Pin = GPIO_PIN_0,
    .Mode = GPIO_MODE_OUTPUT_PP,
    .Pull = GPIO_NOPULL,
    .Speed = GPIO_SPEED_FREQ_LOW
};



typedef struct {
    GPIO_InitTypeDef pin;
    GPIO_TypeDef *port;
    int delay;
    uint8_t counter;
} mygpio_pin;

mygpio_pin pinB0;


int Init_Thread(void) {
    __HAL_RCC_GPIOB_CLK_ENABLE();

    pinB0.pin = led_ld1;
    pinB0.port = GPIOB;
    pinB0.delay = 15;
    pinB0.counter = 1;
    tid_Thread = osThreadNew(Thread, (void *)&pinB0, NULL);
    if (tid_Thread == NULL) return -1;

    return 0;
}

void Thread(void *argument) {
    mygpio_pin *gpio = (mygpio_pin *)argument;
    HAL_GPIO_Init(gpio->port, &(gpio->pin));
    while (1) {
        gpio->counter++;
        HAL_GPIO_TogglePin(gpio->port, gpio->pin.Pin);
        osDelay(gpio->delay);
```

```
    }
}
```

Dependencias

- Librería HAL de STM32.
- CMSIS RTOS v2.

Preguntas y respuestas sobre **ejemplothread**

Esta sección contiene una serie de preguntas con sus respectivas respuestas sobre el funcionamiento del código que utiliza CMSIS RTOS v2 para controlar LEDs en una placa STM32.

Tabla de contenido

¿Qué hace este código? 21

¿Qué es la estructura *mygpio_pin* ? 21

¿Cómo se inicializan los hilos? 21

¿Qué hace la función *Thread()* ? 21

¿Se ejecutan los hilos al mismo tiempo? 22

¿Qué significa *osDelay()* ? 22

¿Qué pasa si *osThreadNew()* devuelve NULL? 22

¿Qué ficheros de cabecera se utilizan? 22

¿Qué hace este código?

Este código crea un hilo (threads) que controla un LED conectado al pin PB0 de una placa STM32F4. El hilo alterna el estado del LED (encendido/apagado) con una frecuencia determinada utilizando funciones del sistema operativo en tiempo real CMSIS RTOS v2. Dentro del codigo del Thread se realiza un casting al tipo de estructura que se utiliza en el ejemplo

¿Qué es la estructura *mygpio_pin* ?

Es una estructura de datos que encapsula la información necesaria para controlar un pin GPIO en este ejemplo:

- `pin` : configuración del pin (tipo, velocidad, modo).
- `port` : puerto GPIO al que pertenece el pin (por ejemplo, GPIOB).
- `delay` : retardo en milisegundos entre cada cambio de estado (toggle).
- `counter` : variable auxiliar que cuenta la cantidad de veces que se ha realizado el toggle.

¿Cómo se inicializan los hilos?

La función `Init_Thread()` habilita el reloj del puerto GPIOB, configura los parámetros de cada LED y crea un hilo con `osThreadNew()` , pasando como argumento la estructura `mygpio_pin` correspondiente a cada LED.

¿Qué hace la función *Thread()* ?

La función `Thread(void *argument)` es ejecutada el hilo. Dentro de ella:

1. Se inicializa el pin GPIO usando `HAL_GPIO_Init` .
2. Se entra en un bucle infinito donde: - Se incermenta el valor de `counter` . - Se cambia el estado del LED con `HAL_GPIO_TogglePin` . - Se espera el tiempo definido en `delay` usando `osDelay` .

¿Se ejecutan los hilos al mismo tiempo?

CMSIS RTOS v2 permite la ejecución concurrente, que no simultanea, de múltiples hilos. El scheduler del sistema operativo se encarga de asignar tiempo de CPU a cada hilo según su estado y prioridad.

¿Qué significa *osDelay()* ?

Es una función del RTOS que suspende la ejecución del hilo actual durante un número determinado de milisegundos. Esto permite que otros hilos se ejecuten mientras tanto. `osDelay` tiene como parametro el número de ticks que la tarea estará bloqueada. El número de ticks por segundo se define en el archivo `RTX_Config.h` (parámetro `Kernel Tick Frequency [Hz]`). En este ejemplo se ha configurado a 1000, por lo que un tick equivale a 1 ms.

¿Qué pasa si *osThreadNew()* devuelve NULL?

Significa que no se pudo crear el hilo. En ese caso, la función `Init_Thread()` devuelve -1 como señal de error.

¿Qué ficheros de cabecera se utilizan?

- `cmsis_os2.h` : para funciones del sistema operativo en tiempo real.
- `stm32f4xx_hal.h` : para funciones de acceso a hardware (HAL).
- `stdlib.h` : para funciones estándar de C.

ejemplothreads : Uso básico de threads en CMSIS RTOS v2

Este documento describe el funcionamiento de un programa en C que utiliza CMSIS RTOS v2 y la biblioteca HAL de STM32 para controlar dos LEDs mediante hilos concurrentes.

Descripción General

El programa crea dos hilos que controlan dos LEDs conectados a los pines PB0 y PB7 del microcontrolador STM32F4. Cada hilo alterna el estado de su LED con una frecuencia distinta, utilizando funciones del sistema operativo en tiempo real (RTOS) y la biblioteca HAL para la configuración y manipulación de los pines GPIO.

Estructura de Datos

Se define una estructura llamada `mygpio_pin` que encapsula toda la información necesaria para controlar un LED:

- `GPIO_InitTypeDef pin` : configuración del pin (modo, velocidad, tipo de salida).
- `GPIO_TypeDef *port` : puerto GPIO al que pertenece el pin.
- `int delay` : retardo en milisegundos entre cada cambio de estado del LED.
- `uint8_t counter` : contador que se alterna en cada iteración del hilo.

Esta estructura permite pasar todos los parámetros necesarios a la función del hilo de forma organizada.

Inicialización de los Hilos

La función `Init_Thread` realiza las siguientes tareas:

1. Habilita el reloj del puerto GPIOB.
2. Configura dos instancias de `mygpio_pin` para los pines PB0 y PB7.
3. Crea dos hilos con `osThreadNew` , cada uno ejecutando la función `Thread` con una instancia diferente de `mygpio_pin` .

Cada hilo se ejecuta de forma independiente y controla su propio LED.

Función del Hilo

La función `Thread` realiza lo siguiente:

1. Inicializa el pin GPIO usando `HAL_GPIO_Init` .
2. Entra en un bucle infinito donde: - Alterna el valor del contador con `~counter` . - Cambia el estado del pin con `HAL_GPIO_TogglePin` . - Espera el tiempo definido en `delay` usando `osDelay` .

Esto provoca que el LED conectado al pin correspondiente parpadee con una frecuencia determinada.

Uso de HAL y CMSIS RTOS

- **HAL (Hardware Abstraction Layer)** : se utiliza para configurar e inicializar los pines GPIO de forma sencilla y portable.
- **CMSIS RTOS v2** : proporciona las funciones para crear y gestionar hilos, como `osThreadNew` y `osDelay` .

Código Fuente

```
#include "cmsis_os2.h"
#include "stm32f4xx_hal.h"
#include <stdlib.h>

osThreadId_t tid_Thread;

GPIO_InitTypeDef led_ld1 = {
    .Pin = GPIO_PIN_0,
    .Mode = GPIO_MODE_OUTPUT_PP,
    .Pull = GPIO_NOPULL,
    .Speed = GPIO_SPEED_FREQ_LOW
};

GPIO_InitTypeDef led_ld2 = {
    .Pin = GPIO_PIN_7,
    .Mode = GPIO_MODE_OUTPUT_PP,
    .Pull = GPIO_NOPULL,
    .Speed = GPIO_SPEED_FREQ_LOW
};

typedef struct {
    GPIO_InitTypeDef pin;
    GPIO_TypeDef *port;
    int delay;
    uint8_t counter;
} mygpio_pin;

mygpio_pin pinB0;
mygpio_pin pinB7;

int Init_Thread(void) {
    __HAL_RCC_GPIOB_CLK_ENABLE();

    pinB0.pin = led_ld1;
    pinB0.port = GPIOB;
    pinB0.delay = 15;
    pinB0.counter = 1;
    tid_Thread = osThreadNew(Thread, (void *)&pinB0, NULL);
    if (tid_Thread == NULL) return -1;

    pinB7.pin = led_ld2;
    pinB7.port = GPIOB;
    pinB7.delay = 10;
    pinB7.counter = 0;
    tid_Thread = osThreadNew(Thread, (void *)&pinB7, NULL);
```

```
    if (tid_Thread == NULL) return -1;

    return 0;
}

void Thread(void *argument) {
    mygpio_pin *gpio = (mygpio_pin *)argument;
    HAL_GPIO_Init(gpio->port, &(gpio->pin));
    while (1) {
        gpio->counter++;
        HAL_GPIO_TogglePin(gpio->port, gpio->pin.Pin);
        osDelay(gpio->delay);
    }
}
```

Dependencias

- Librería HAL de STM32.
- CMSIS RTOS v2.

Preguntas y respuestas sobre **ejemplothreads**

Esta sección contiene una serie de preguntas con sus respectivas respuestas sobre el funcionamiento del código que utiliza CMSIS RTOS v2 para controlar LEDs en una placa STM32.

Tabla de contenido

¿Qué hace este código? 21

¿Qué hace la función *Thread()* ? 21

¿Se ejecutan los hilos al mismo tiempo? 22

¿Qué pasa si *osThreadNew()* devuelve NULL? 22

¿Qué ficheros de cabecera se utilizan? 22

¿Qué hace este código?

Este código crea dos hilos (threads) que controlan dos LEDs conectados a los pines PB0 y PB7 de una placa STM32F4. Cada hilo alterna el estado del LED (encendido/apagado) con una frecuencia determinada utilizando funciones del sistema operativo en tiempo real CMSIS RTOS v2. Es importante entender que el mismo codigo (funcion Thread) es ejecutado por dos hilos diferentes, cada uno con sus propios parámetros, que se reciben en el argumento de la función. Es de tipo `void` para poder pasar cualquier tipo de estructura como argumento. Dentro del codigo del Thread se realiza un casting al tipo de estructura que se utiliza en el ejemplo

Es una estructura de datos que encapsula la información necesaria para controlar un pin GPIO en este ejemplo:

- `pin` : configuración del pin (tipo, velocidad, modo).
- `port` : puerto GPIO al que pertenece el pin (por ejemplo, GPIOB).
- `delay` : retardo en milisegundos entre cada cambio de estado (toggle).
- `counter` : variable auxiliar que cuenta la cantidad de veces que se ha realizado el toggle.

La función `Init_Thread()` habilita el reloj del puerto GPIOB, configura los parámetros de cada LED y crea dos hilos con `osThreadNew()` , pasando como argumento la estructura `mygpio_pin` correspondiente a cada LED.

¿Qué hace la función *Thread()* ?

La función `Thread(void *argument)` es ejecutada por cada hilo. Dentro de ella:

1. Se inicializa el pin GPIO usando `HAL_GPIO_Init` .
2. Se entra en un bucle infinito donde: - Se alterna el valor de `counter` . - Se cambia el estado del LED con `HAL_GPIO_TogglePin` . - Se espera el tiempo definido en `delay` usando `osDelay` .

¿Se ejecutan los hilos al mismo tiempo?

CMSIS RTOS v2 permite la ejecución concurrente, que no simultanea, de múltiples hilos. El scheduler del sistema operativo se encarga de asignar tiempo de CPU a cada hilo según su estado y prioridad.

Es una función del RTOS que suspende la ejecución del hilo actual durante un número determinado de milisegundos. Esto permite que otros hilos se ejecuten mientras tanto. `osDelay` tiene como parametro el número de ticks que la tarea estará bloqueada. El número de ticks por segundo se define en el archivo `RTX_Config.h` (parámetro `Kernel Tick Frequency [Hz]`). En este ejemplo se ha configurado a 1000, por lo que un tick equivale a 1 ms.

¿Qué pasa si *osThreadNew()* devuelve NULL?

Significa que no se pudo crear el hilo. En ese caso, la función `Init_Thread()` devuelve -1 como señal de error.

¿Qué ficheros de cabecera se utilizan?

- `cmsis_os2.h` : para funciones del sistema operativo en tiempo real.
- `stm32f4xx_hal.h` : para funciones de acceso a hardware (HAL).
- `stdlib.h` : para funciones estándar de C.

ejemplothreads-queues : Uso básico de threads y colas en CMSIS RTOS v2

Este documento describe el funcionamiento de un programa en C que utiliza CMSIS RTOS v2 y la biblioteca HAL de STM32 para controlar dos LEDs mediante hilos concurrentes que se comunican con colas.

Descripción General

El programa crea dos hilos, denominados `Producer` y `Consumer` . El hilo `Producer` se encarga de introducir datos en la una cola que tiene el identificador `id_MsgQueue` . El numero total de mensajes que se introducen en la cola en cada iteración del bucle `while` es 32 con un tiempo de retardo entre operaciones de escritura que es variable El `Consumer` se encarga de extraer los datos de cola y de actuar sobre los leds en función del valor leido de la cola.

Estructura de Datos

Se define una estructura llamada `mygpio_pin` que encapsula toda la información necesaria para controlar un LED:

- `GPIO_InitTypeDef pin` : configuración del pin (modo, velocidad, tipo de salida).
- `GPIO_TypeDef *port` : puerto GPIO al que pertenece el pin.

Esta estructura permite pasar todos los parámetros necesarios a la función del hilo de forma organizada.

Inicialización de los Hilos

La función `Init_Thread` realiza las siguientes operaciones:

1. Crea una cola con `osMessageQueueNew` para almacenar hasta 16 mensajes cada uno de tamaño un `uint8_t` , es decir, un solo byte.
2. Crea un hilo `Consumer` con `osThreadNew` , ejecutando la función `Consumer` .
3. Crea un hilo `Producer` con `osThreadNew` , ejecutando la función `Producer` .

Función del Hilo Producer

La función `Producer(void *argument)` realiza las siguientes operaciones: 1. De manera continua en un bucle infinito inserta en cola en cada iteración del bucle while 32 mensajes de 1 byte con el valor de la variable index. 2. Los mensajes se introducen con un retardo que varia desde 100ms hasta 400ms 3. La función osMessageQueuePut introduce los mensajes con timeout 0, lo cúal implica que si no hay sitio en la cola el mensaje no se podrá guardar.

Función del Hilo Consumer

La función `Consumer(void *argument)` realiza las siguientes operaciones: 1. De manera continua en un bucle infinito extrae de la cola los mensajes introducidos por el hilo `Producer` . 2. Si se saca un valor de la cola se procede a encender o apagar los leds en función del valor leido. 3. La variable errors_or_timeout cuenta el número de veces que no se ha podido leer un mensaje de la cola, ya sea por timeout o porque la cola está vacía.

Uso de HAL y CMSIS RTOS

- **HAL (Hardware Abstraction Layer)** : se utiliza para configurar e inicializar los pines GPIO de forma sencilla y portable.
- **CMSIS RTOS v2** : proporciona las funciones para crear y gestionar hilos, como `osThreadNew` y `osDelay` .

Código Fuente

```
#include "cmsis_os2.h"                          // CMSIS RTOS header
file
#include "stm32f4xx_hal.h"
#include <string.h>
#include <stdlib.h>


osThreadId_t tid_Thread;                        // thread id
osMessageQueueId_t id_MsgQueue;
int Init_Thread (void);
void Producer (void *argument);                   // thread function
producing data
void Consumer (void *argument);                   // thread function
consuming data
int qsize=0;
uint16_t h=0;
uint8_t i=0;

typedef struct  {
        GPIO_InitTypeDef pin;
                GPIO_TypeDef *port;
} mygpio_pin;

mygpio_pin pinB0;
mygpio_pin pinB7;

int Init_Thread (void) {

        id_MsgQueue = osMessageQueueNew(16, sizeof(uint8_t), NULL);


tid_Thread = osThreadNew(Producer, NULL, NULL);
if (tid_Thread == NULL) {
        return(-1);
}

        tid_Thread = osThreadNew(Consumer, NULL, NULL);
if (tid_Thread == NULL) {
        return(-1);
}

return(0);
}

void Producer (void *argument) {
```

```
        uint8_t index=0;
        osStatus_t status;
while (1) {
                for( h=1; h<5; h++){
                        for( i=0; i< 8; i++){

status=osMessageQueuePut(id_MsgQueue, &index, 0U, 0U);
                                index++;
                                osDelay(h*100);
                        }
                }
        }
}
void Consumer (void *argument) {
        uint8_t val=0;
        osStatus_t status;
        int errors_or_timeouts=0;
        GPIO_InitTypeDef led_ld1 = {
                .Pin = GPIO_PIN_0,
                .Mode = GPIO_MODE_OUTPUT_PP,
                .Pull = GPIO_NOPULL,
                .Speed = GPIO_SPEED_FREQ_LOW
        };
        GPIO_InitTypeDef led_ld2 = {
                .Pin = GPIO_PIN_7,
                .Mode = GPIO_MODE_OUTPUT_PP,
                .Pull = GPIO_NOPULL,
                .Speed = GPIO_SPEED_FREQ_LOW
        };
        __HAL_RCC_GPIOB_CLK_ENABLE();

        HAL_GPIO_Init(GPIOB, &led_ld1);

        HAL_GPIO_Init(GPIOB, &led_ld2);


while (1) {
        qsize=osMessageQueueGetCount (id_MsgQueue);
                status = osMessageQueueGet(id_MsgQueue, &val, NULL,
10U);   // wait for message
                if (status == osOK){
                        HAL_GPIO_WritePin(GPIOB,led_ld1.Pin,
(GPIO_PinState) val&0x01);
                        HAL_GPIO_WritePin(GPIOB,led_ld2.Pin,
(GPIO_PinState)(val&0x02)>>1);

                }
                else {
                        errors_or_timeouts++;
                }
                osDelay(250); //This delay is to simulate an
operation that needs 101ms to complete
```

```
}
}
```

Dependencias

- Librería HAL de STM32.
- CMSIS RTOS v2.

Preguntas y respuestas sobre **ejemplothreads-queues**

Esta sección contiene una serie de preguntas con sus respectivas respuestas sobre el funcionamiento del código que utiliza CMSIS RTOS v2 para controlar LEDs en una placa STM32.

Tabla de contenido

¿Cuál es el propósito de la cola de mensajes *id_MsgQueue* en esta aplicación? 46

¿Qué función cumple el bucle anidado en el hilo *Producer* ? 47

¿Cuanto tiempo tarda en llenarse la cola de mensajes *id_MsgQueue* ?47

¿cuanto vale la variable errors_or_timeouts despues de 1 minuto de ejecución del código? 47

¿Cuál es el propósito de la cola de mensajes *id_MsgQueue* en esta aplicación?

La cola de mensajes *id_MsgQueue* actúa como un canal de comunicación y sincronización entre los hilos *Producer* y *Consumer* . Permite que el hilo productor envíe datos (índices) al consumidor de forma segura y sincronizada. Al definir una cola con capacidad para 16 elementos de tipo *uint8_t* , se establece un buffer temporal que desacopla la producción y el consumo de datos.

¿Qué función cumple el bucle anidado en el hilo *Producer* ?

El bucle anidado en *Producer* genera una secuencia de valores que se colocan en la cola de mensajes. El bucle externo recorre *h* de 1 a 4, y el interno recorre *i* de 0 a 7. En cada iteración, se coloca un valor en la cola (*index*) y se incrementa. El retardo *osDelay(h*100)* introduce una variabilidad en el tiempo entre envíos, oscilando entre 100 ms y 400 ms. Esto simula diferentes tasas de producción de datos.

¿Cuanto tiempo tarda en llenarse la cola de mensajes *id_MsgQueue* ?

En la cola se introducen 32 mensajes en cada ciclo completo de los bucles anidados (8 mensajes por cada uno de los 4 valores de *h*) pero el Thread Consumer extrae mensajes cada 250ms en el caso de que existan. Por tanto la cola nunca llega a llenarse. Intente calcular cual sería el numero máximo de mensajes que se pueden acumular en la cola.

¿cuanto vale la variable errors_or_timeouts despues de 1 minuto de ejecución del código?

l valor vale 0 porque no se produce dicha condición.

Note

Challenge: Modifique el código del hilo `Producer` para que la variable errors_or_timeouts no valga cero.

ejemplothreads-flags : Uso básico de threads y flags en CMSIS RTOS v2

Este documento describe el funcionamiento de un programa en C que utiliza CMSIS RTOS v2 y la biblioteca HAL de STM32 para controlar dos LEDs mediante hilos concurrentes que se comunican con colas.

Descripción General

El programa crea dos hilos, denominados `Producer` y `Consumer` . El hilo `Producer` es un bucle infinito que se encarga de activar flags para el thread `Consumer` . Este en función de los flags activados ejecuta unas acciones u otras.

Estructura de Datos

Se define una estructura llamada `mygpio_pin` que encapsula toda la información necesaria para controlar un LED:

- `GPIO_InitTypeDef pin` : configuración del pin (modo, velocidad, tipo de salida).
- `GPIO_TypeDef *port` : puerto GPIO al que pertenece el pin.

Esta estructura permite pasar todos los parámetros necesarios a la función del hilo de forma organizada.

Inicialización de los Hilos

La función `Init_Thread` realiza las siguientes operaciones:

1. Crea una cola con `osMessageQueueNew` para almacenar hasta 16 mensajes cada uno de tamaño un `uint8_t` , es decir, un solo byte.
2. Crea un hilo `Consumer` con `osThreadNew` , ejecutando la función `Consumer` .
3. Crea un hilo `Producer` con `osThreadNew` , ejecutando la función `Producer` .

Función del Hilo Producer

La función `Producer(void *argument)` realiza las siguientes operaciones: 1. De manera continua en un bucle infinito señaliza flags en el thread `Consumer` . 2. Los flags activados son el 0x0001 y en 0x0002.

Función del Hilo Consumer

La función `Consumer(void *argument)` realiza las siguientes operaciones: 1. Inticializa dos pines del GPIO en el puerto B. 2. Espera de manera infinita (`osWaitForEver`) a que cualquiera (`osFlagsWaitAny`) de los flags (`0` o `1` , en hexadecimal `0x03`) se activen. 3. Si el flag activado es el `0` se hace un toggle en el pin 0 del GPIOB. Si el activado es el `1` se hace el toggle en el pin 7 del GPIOB. 4. Si se produce otra condición se incrementa la variable `errors` .

Uso de HAL y CMSIS RTOS

- **HAL (Hardware Abstraction Layer)** : se utiliza para configurar e inicializar los pines GPIO de forma sencilla y portable.
- **CMSIS RTOS v2** : proporciona las funciones para crear y gestionar hilos, como `osThreadNew` y `osDelay` .

Código Fuente

```
#include "cmsis_os2.h"                          // CMSIS RTOS header
file
#include "stm32f4xx_hal.h"
#include <string.h>
#include <stdlib.h>


osThreadId_t tid_Thread_producer;                       // thread id
osThreadId_t tid_Thread_consumer;
int Init_Thread (void);
void Producer (void *argument);                   // thread function
producing data
void Consumer (void *argument);                   // thread function
consuming data
int qsize=0;
uint8_t a=0;
uint8_t b=0;

typedef struct  {
        GPIO_InitTypeDef pin;
                GPIO_TypeDef *port;
} mygpio_pin;

mygpio_pin pinB0;
mygpio_pin pinB7;

int Init_Thread (void) {




tid_Thread_producer = osThreadNew(Producer, NULL, NULL);
if (tid_Thread_producer == NULL) {
        return(-1);
}

        tid_Thread_consumer = osThreadNew(Consumer, NULL, NULL);
if (tid_Thread_consumer == NULL) {
        return(-1);
}

return(0);
}

void Producer (void *argument) {
```

```
        uint32_t status;
while (1) {


                                status=
osThreadFlagsSet(tid_Thread_consumer,0x0001);
                                osDelay(1000);
                                status=
osThreadFlagsSet(tid_Thread_consumer,0x0002);
                                osDelay(1000);

        }
}
void Consumer (void *argument) {
        uint8_t val=0;
        uint32_t status;
        int errors=0;
        GPIO_InitTypeDef led_ld1 = {
                .Pin = GPIO_PIN_0,
                .Mode = GPIO_MODE_OUTPUT_PP,
                .Pull = GPIO_NOPULL,
                .Speed = GPIO_SPEED_FREQ_LOW
        };
        GPIO_InitTypeDef led_ld2 = {
                .Pin = GPIO_PIN_7,
                .Mode = GPIO_MODE_OUTPUT_PP,
                .Pull = GPIO_NOPULL,
                .Speed = GPIO_SPEED_FREQ_LOW
        };
        __HAL_RCC_GPIOB_CLK_ENABLE();

        HAL_GPIO_Init(GPIOB, &led_ld1);

        HAL_GPIO_Init(GPIOB, &led_ld2);


while (1) {
        status=osThreadFlagsWait(0x3,osFlagsWaitAny,osWaitForever);
                switch (status){
                        case 1:

HAL_GPIO_TogglePin(GPIOB,led_ld1.Pin);
                                a=!a;
                                break;
                case 2:

HAL_GPIO_TogglePin(GPIOB,led_ld2.Pin);
                        b=!b;
                                break;
                default:errors++;
                                break;
```

```
                    }


            }
}
```

Dependencias

- Librería HAL de STM32.
- CMSIS RTOS v2.

Preguntas y respuestas sobre **ejemplothreads-flags**

Esta sección contiene una serie de preguntas con sus respectivas respuestas sobre el funcionamiento del código que utiliza CMSIS RTOS v2 para controlar LEDs en una placa STM32.

Tabla de contenido

Se modifica el código del Producer para que envíe ambas señales (0x0001 y 0x0002) de forma casi simultánea, seguido de un delay de 1 segundo: 59

Se modifica el código del Producer para que envíe ambas señales (0x0001 y 0x0002) de forma casi simultánea, seguido de un delay de 1 segundo:

```
1 void Producer (void *argument) {
2         uint32_t status;
3         while (1) {
4                 status = osThreadFlagsSet(tid_Thread_consumer,
0x0001);
5                 status = osThreadFlagsSet(tid_Thread_consumer,
0x0002);
6                 osDelay(1000);
7         }
8 }
```

Analice el comportamiento resultante del sistema y responda:

1. ¿Qué valor tendría la variable status en el Consumer después de osThreadFlagsWait?

2. ¿Cómo afecta esta modificación al parpadeo de los LEDs?

1. Valor de status: La variable status en el Consumer tendría el valor 0x0003 (0x0001 | 0x0002), ya que los flags se acumulan en el sistema CMSIS-RTOS cuando se envían antes de que el thread destino los procese.

2. Efecto en los LEDs: Los LEDs dejarían de parpadear por completo. El switch statement en el Consumer solo maneja explícitamente los casos 1 (0x0001) y 2 (0x0002). Al recibir el valor combinado 3, la ejecución cae en el caso default, donde solo se incrementa la variable errors sin ejecutar ninguna operación de toggle en los GPIOs.

CMSIS RTOS V2 Usage Examples

The repository contains basic examples to understand how the CMSIS-RTOS V2 API works using the RTX version 5 operating system. The examples are implemented to make minimal use of the microcontroller peripherals and to emphasize operating system concepts.

The examples have been implemented for the STM32F429 used in the course **Microprocessor-Based Systems** of the **(ETSI Sistemas de Telecomunicación) Universidad Politécnica de Madrid** , and they can be executed using the microprocessor simulator included in the ARM Keil Microvision environment or directly on the hardware.

Code download

To download the code you can use a git client on your computer or download the entire repository (in **zip** format). The instructions to clone the repository are:

Note

Code download

```
$ git clone https://github.com/mruizglz/SBM-rtos.git
```

List of included examples

Included examples

Folder	Goals
ejemplothreads	Learn the basics of creating threads. Use of the same function with parameters to create multiple threads
ejemplothreads-flags	Thread synchronization using flags
ejemplothreads-queues	Data exchange between threads using queues
ejemplothreads-timers	Management of software timers

Keil Project Configuration

Using the simulator

ARM Keil Microvision provides options to configure where the application will run (icon *Options for Target*). Select **Debug** and enable the simulator (**Use Simulator**). You need to configure the **Initialization File** so that it loads a microcontroller configuration script. In this case, select the file `simulator.ini` found in each example folder.

For example, in the `.\ejemplothreads` folder of the repository you will find the file **simulator.ini** with this content:

```
MAP 0x40000000,0x400FFFFF read write
MAP 0xE0000000,0xE00FFFFF read write
```

The meaning of these instructions is to enable, for the simulator, read/write operations in the memory areas where the peripherals are located.

Tip

When using the Keil simulator, write and read operations on peripherals have no effect and therefore you will not be able to simulate their hardware behavior. All operations of the HAL layer that act on peripherals will have no effect. For example, if the code configures a pin as output and then writes a high level to it, you will not see any change in the pin state.

As you can see in the `main.c` program code, there is conditional compilation to include or not the RCC configuration code to use an external clock (HSE). If you use the simulator you must disable this option and use the internal clock (HSI), which is what the simulator uses. The **C/C++(AC6)** tab allows you to add labels in **define** . Include `SIMULATOR` if you want to use the simulator.

Using the hardware

If you have a board with the STM32F429 microcontroller you can run the code directly on the hardware. In this case you must configure the project options so that it uses **ST-Link** instead of the simulator.

Do not define the `SIMULATOR` variable in the compilation options so that the RCC circuit is configured properly.

Debugging applications using CMSIS-RTOS V2

Debugging should be performed by combining the use of breakpoints and the **RTX RTOS** view available in the menu `View->Watch Windows->RTX RTOS` . This allows you to see the state of the different operating system objects when the processor pauses its execution. Complementary tools to understand how an application works are `Logic Analyzer` , `Performance Analyzer` , `System Analyzer` , `Event Recorder` , `Event Statistics` and `Symbols Window` .

Symbols Window

The **Symbols Window** option allows you to view and explore all the symbols defined in the project, including global variables, static variables, functions and register addresses. This window is useful for real-time debugging and analysis.

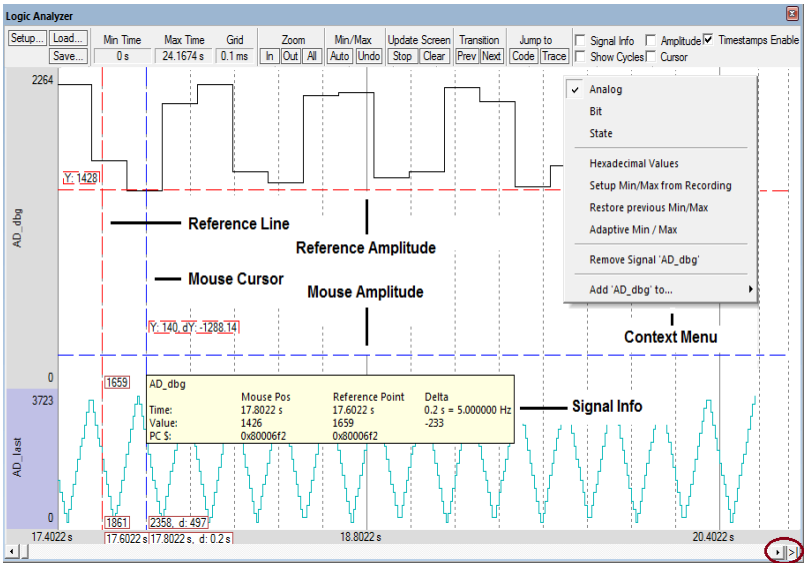
- Displays a hierarchical list of all symbols available in the loaded program.
- Allows searching and filtering symbols by name.
- Shows the address and current value of each symbol during the debugging session.
- Facilitates dragging variables to other analysis windows, such as the Watch Window or the Logic Analyzer.
- Allows examining optimized variables if they are available in the symbol table.
- If a symbol does not appear, verify the compiler optimization settings and the variable scope.

To use it:

1. Start a debugging session.
2. Open the window from the menu: View ▸ Symbol Window .
3. Search for the desired symbol using the filter field.
4. Drag the symbol to the Watch window or the `Logic Analyzer` for monitoring.

Logic Analyzer

It allows you to visualize the time evolution of variables that are global to the application, the content of memory locations, etc. You can configure the value range and it is very useful to visually compare the evolution of the software application by tracking variables. To add signals to the `Logic Analyzer` you can drag them from the Symbol window or type the signal name.



ARM Keil Microvision Logic Analyzer.

© Copyright UPM.
Created using Sphinx 8.2.3.